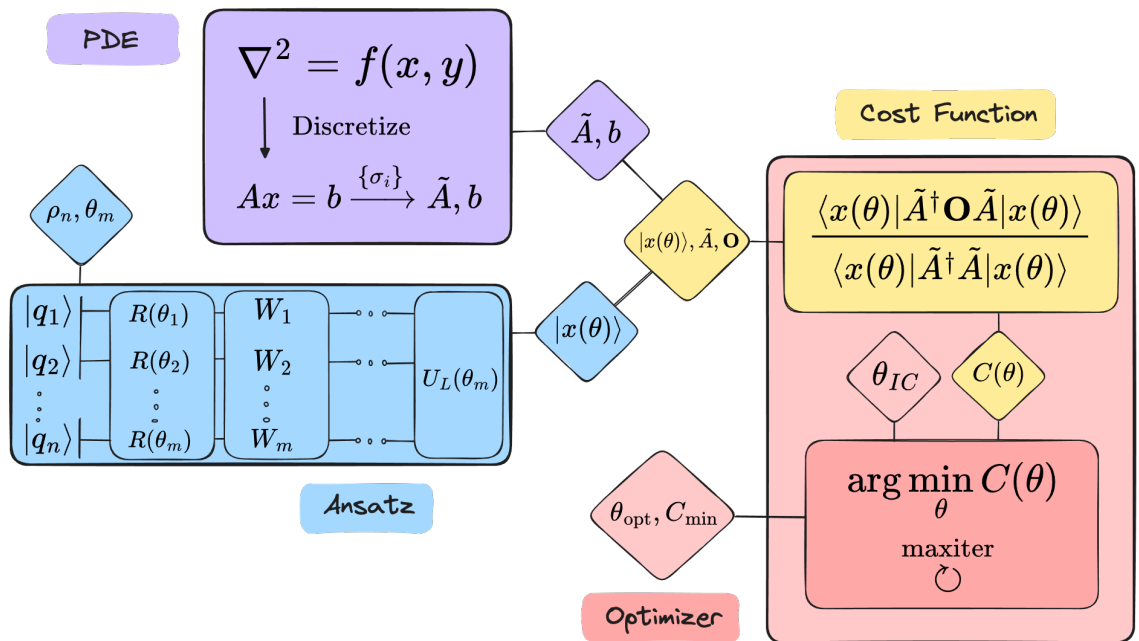


VQLS Basics



Oliver Jackson

July 28, 2026

Contents

1	Introduction	1
2	Variational quantum linear solvers	3
2.1	Variational quantum algorithms	3
2.2	Quantum states	6
2.3	The variational principle	8
2.4	The heart of the VQLS: The parameterized quantum circuit	9
3	Building a VQLS	12
3.1	Constructing the operators	13
3.2	Choosing an ansatz	14
3.3	Building the cost function	16
3.4	Choosing an optimizer	19
4	The readout problem to be continued	21

Chapter 1

Introduction

This document covers variational quantum linear solvers (VQLS) where the goal is to take a sparse matrix A and a vector b , and prepare a quantum state $|x\rangle$ that is proportional to $A^{-1}|b\rangle$.

I hope the reader is familiar with undergraduate linear algebra, differential equations, and dirac notation. Even so, I would encourage the inexperienced reader to dive into the document. I have provided lots of diagrams, equations, and graphs to help aid the learning process.

Note if your goal is to see a "quantum advantage" by solving the Poisson problem using a VQLS then this project is not for you. In the end, this project amounts classically to a small to medium sized matrix multiplication which can be handled by your smart phone in a matter of milliseconds to seconds. A true advantage may be proposed for quantum systems or large system sizes in which a classical high performance computer cannot handle. However, I am interested in solving PDEs right now on noisy quantum hardware. So my research is more concerned with improving the scalability and runtime issues of VQLS on feasible system sizes.

This document was heavily inspired by "Introduction to Variational Quantum Algorithms" by Michał Stechly and "Variational quantum algorithms" by Marco Cerezo et al. Michał Stechly provides an easily digestible introduction to VQA's which can give a great base for those coming from non-technical backgrounds. Cerezo's paper on VQA's and provides a comprehensive overview on the class of algorithms and their variations. This

document is not a replacement for those two papers but rather a supplement to them for those specifically interested in solving PDEs with VQLS. I spend more time on each component of the algorithm introducing both the concept and the application.

Chapter 2

Variational quantum linear solvers

The VQLS is a hybrid quantum-classical algorithm that uses a parameterized quantum circuit (PQC) to iteratively prepare a quantum state $|x\rangle$ that is proportional to $A^{-1}|b\rangle$. VQLS uses both a quantum processing unit (QPU) and a CPU. It also depends on the primitive PQC which is a series of unitary gates where at least some of the gates depend on parameters θ . To find a solution we apply a PQC to a set of quantum states which prepares a guess for $|x\rangle$. We apply the laplacian against the guess which returns an approximation to the driving function. Then we check how much this approximation overlaps with the original driving function and then update the parameters to make a better guess. You can think of VQLS as a quantum analog to classical machine-learning methods like neural networks. This is a good analogy because the PQC is like an interconnected network with nodes and also has a training loop which uses an optimizer. If this was confusing that is ok. In the following sections I break down the definitions and context of each of these concepts. You can refer back to this definition as you read along.

2.1 Variational quantum algorithms

VQLS is a type of variational quantum algorithm (VQA) which are a class of algorithms designed to solve problems on noisy intermediate scale quantum (NISQ) devices. VQAs account for the major limitations of NISQ poor

coherence times and gate errors by utilizing a learning based approach. Like VQAs they also have a PQC and optimizer but are not limited to solving linear systems of equations.

VQAs can be used for a wide variety of problems including optimization, PDEs, and quantum chemistry. Of course, whether VQAs can actually provide good solutions to problems on quantum hardware is still an open question. I am investigating whether they can be used to efficiently solve linear systems of equations. The more we research the more we learn about the specific limitations of each application. We are also finding that VQAs are not very capable of solving problems on NISQ devices right now but still have potential to be improved. For example, creating ansatz which are problem specific, optimizers which are robust to noise, and cost functions which are less susceptible to barren plateaus are all active areas of research.

All of the accessible quantum hardware to date is noisy and has a limited qubit coherence time and we refer to this hardware as NISQ. On IBM's quantum hardware, for example, applying one gate to a qubit introduces an error of about 0.1% to 1%. This means that if we apply a series of gates to a qubit, the error compounds quickly and the qubit will lose its quantum state after only a hundred gates. This limits the depth (number of quantum gates applied) of quantum circuits that can be executed on NISQ devices.

An algorithm that can work with shallow circuits is necessary. The best known quantum algorithms for solving linear systems are the HHL and QSVT algorithms which use the primitives quantum phase estimation (QPE) and quantum singular value transformation (QSVT) respectively. Both of these primitives require deep circuits and are not suitable for NISQ devices which is why I have not included them in this project. In interest of seeing if we can solve PDEs on quantum devices today using IBM's hardware we will focus on the VQLS algorithm.

So far we know VQAs are a class of algorithms for NISQ that can solve many problems. But how do they work? How is a VQA constructed? VQAs can be described using four steps.

1. Encode the problem
2. Choose the ansatz (a kind of PQC)
3. Build the cost function

4. Run the optimization loop

The final step which is hidden is to readout the solution. However, this can be a very difficult step and is not always possible. I will discuss general readout methods and their limitations in the last chapter. Encoding the problem is the first step and is when we take our classical problem and encode it into a quantum problem. In quantum chemistry problems we encode the Hamiltonian of a molecule into a quantum circuit. In the case of a PDE we can encode a matrix A and vector b containing the discretized equations into a quantum circuit.

For example, we can take any general classical problem represented by the 2x2 matrix

$$H = \begin{bmatrix} a & b \\ c & c \end{bmatrix}.$$

Unlike classical computers, which can store arbitrary matrices as grids of numbers in memory, quantum computers only receive instructions through a well formulated inner product space. So in order to encode this matrix we have to decompose it into an orthonormal basis. The Pauli matrices form an orthonormal basis for 2x2 matrices and consists of the identity matrix and three Pauli matrices

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad P_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad P_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad P_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

We could use any orthonormal basis to decompose our matrix into but I have chosen Pauli matrices because they are native to IBM's quantum hardware. We can define a new matrix H that is a linear combination of Pauli matrices.

$$H = c_1 X + c_2 Y + c_3 Z + c_4 I$$

Where the coefficients c_k are determined by the equation $\frac{1}{2} \text{Tr}(A \cdot \sigma_k)$. The ansatz is the specific architecture you use for your PQC. Mathematically the ansatz is a series of parameterized and unparameterized unitary gates applied to a quantum state. And it controls how guesses are made for your solution

state. In the PQC section and the ansatz section I discuss exactly how the ansatz prepares a guess solution.

After choosing our ansatz we need define our cost function. The cost function takes in our encoded matrix as well as the ansatz. We define a cost function like in classical machine learning methods to be the discrepancy between our guessed state and our target state. The final output of the cost function is a real number between zero and one. The core goal of the VQA is to minimize this cost through optimization. The final step is to choose an optimizer type like gradient descent.

2.2 Quantum states

Before jumping into the PQC I want to introduce quantum states and the variational principle to give some motivation to the reader on why this algorithm works. Very important. On a classical computer an n -bit register lives in a discrete space where each bit can be exactly zero or one. In quantum computing, we take the discrete classical states a system can occupy (such as 00,01,10 or 11), make each one an orthogonal axis, and assign a complex number to each one. A single quantum state is simply a continuous vector pointing somewhere inside this space.

$$|\psi\rangle = \sum_{x=0}^{2^n-1} c_x |x\rangle \quad \text{where} \quad |\psi\rangle \in \mathbb{C}^{2^n}$$

So now our quantum state which is a vector can be formed with any linear combination of possible states and their corresponding complex coefficients. As long as the sum of coefficients squared magnitudes is equal to exactly one. This forms a vector space or more specifically an inner product space commonly referred to as the Hilbert space. Therefore, you can mathematically encode 2^n classical data points into the coefficients of a single n -qubit quantum state. Confirm that this makes sense for yourself.

If a quantum state is a vector pointing somewhere in a continuous complex space, then a quantum logic gate is simply a mathematical operation that rotates or transforms that vector. It is a unitary transformation which means it preserves the inner product (the length) of the state vector. This ensures

the total probability of all possible outcomes remains exactly one. A transformation is unitary if its conjugate transpose times itself returns the identity matrix

$$U^\dagger U = I.$$

So a quantum gate can be mathematically represented as a unitary transformation matrix. Let's take the example of an X gate being applied to one qubit in the zero state. The X gate is represented by the Pauli x matrix and it takes the qubit state and flips it. So the transformation is given by

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} |0\rangle = |1\rangle.$$

This is a perfect explanation for what is happening algorithmically, however, it is easy for beginners to confuse actual quantum gates with matrices. On real hardware a transformation applied to a qubit by injecting a very specific amount of energy into the qubit (such as a photon or electron) which is extremely difficult to do for systems with more than one particle. This is why many quantum computing software packages, like qiskit `Statevector`, have support for qubit simulation. In `Statevector` the qubits are manipulated using pauli matrices through matrix multiplication. Then by changing our dependence to real hardware like `ibm_boston` those same matrices are compiled and translated into machine instructions.

These instructions are described by the time-dependent schrodinger equation. It is not necessary to understand what it does. But notice that there is a dependence on time, and that this equation describes how a quantum state evolves using the transformation Hamiltonian H

$$i\hbar \frac{d}{dt} |\Psi(t)\rangle = H |\Psi(t)\rangle.$$

When this equation is solved it has the form

$$|\Psi(t)\rangle = e^{-iHt} |\Psi(0)\rangle.$$

This is telling us that any quantum state evolves by taking an initial state and act on it by applying the transformation through e^{-iHt} we will get a final state. Now using a similar form we can describe any unitary matrix as an imaginary exponential of a Hermitian matrix H

$$U = e^{-iH}.$$

Note that we no longer have dependence on time. Any quantum circuit is built by sequentially applying gates to a qubit register. Because we know gates are just unitary matrices, a quantum circuit is just the product of these unitary gates

$$\text{QC} = \prod_{l=1}^L U_l$$

where l is the number of layers.

2.3 The variational principle

The variational principle will help us understand why VQAs work. In nature, quantum states are described by the schrodinger equation. And the variational principle seeks out only stationary states. A stationary state is an energy eigenstate where the energy remains constant. Now take, for example, a single particle in a stationary state. Its energy is described by the time-independent form of the schrodinger equation, like so

$$\hat{H}|\psi\rangle = E|\psi\rangle.$$

This is also called the eigenvalue equation in linear algebra. We can find the energy of a specific eigenstate by applying the Hamiltonian H

$$\hat{H}|\psi_\lambda\rangle = \lambda|\psi_\lambda\rangle.$$

We might be interested in the lowest energy state (the ground state) denoted by

$$\hat{H}|\psi_0\rangle = E_0|\psi_0\rangle.$$

Once we have the ground state we can determine lots of useful things about our system like its physical or chemical properties. However, the ground state is not limited to nature and can be reformulated to represent the solution of problems in optimization, economics, or PDE solving. We can extract the solution by multiplying the left side with $\langle\psi|$. This is the formula for an expectation value of H with respect to $|\psi_0\rangle$. It represents the expected value of the ground state energy.

$$\langle\psi_0|\hat{H}|\psi_0\rangle = E_0$$

However, we usually don't know what the specific eigenstates $|\psi_\lambda\rangle$ the particle can take on. Because the ground state has the absolute lowest energy eigenvalue we know any arbitrary state must have an energy that is larger than the ground state.

$$\langle\psi|\hat{H}|\psi\rangle \geq E_0$$

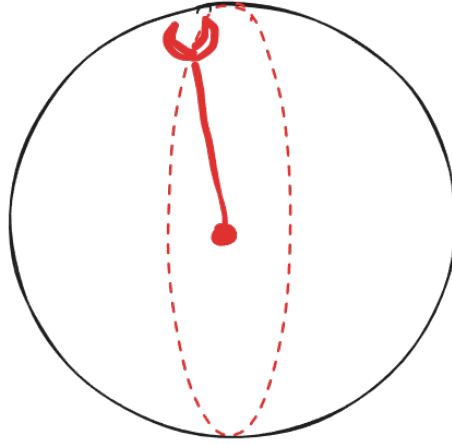
This equation is called the variational principle. It says that the expectation value of a Hamiltonian must be larger than or equal to the ground state energy. We have a system that can take on a whole space of states, we are looking for a specific eigenstate within this space to find its eigenvalue. Now we just need to shuffle through the whole space of potential eigenstates right? For large system this can be like finding a needle in a haystack. Instead, we need a tool that will help us quickly explore the space of good eigenstates which bring us close to the ground state. And that does not waste any time trying out poor solutions that don't bring us close to the ground state. This is where the PQC comes in handy.

2.4 The heart of the VQLS: The parameterized quantum circuit

A PQC is the same as a quantum circuit except it has parameters

$$\text{PQC} = \prod_{l=1}^L U_l(\theta).$$

How we define or select these parameters will determine how the PQC explores potential eigenstates. Let's say that our particle from earlier only has real eigenstates. We know that a quantum state lives in complex space. So it would be efficient to limit that space to only have real values which our solution is also restricted to. Think of this space as a sphere with a robot arm attached in the middle. The robot arm can only reach segments of the spheres surface based on what gears we give it. Each gear is like a degree of freedom allowing the robot arm to cover more area along the sphere. If we give the robot arm a gear for an arbitray axis then it could reach all points along that axis.



We can give that freedom to our PQC by applying the Pauli matrices from earlier. They will act as our gears. To achieve similar rotation that the robot arm above does we can use the Pauli y matrix from earlier to formulate a quantum gate given by

$$R_y(\theta) = e^{-i\frac{\theta}{2}Y}.$$

Using this form we can construct a simple one qubit PQC

$$e^{-i\frac{\theta}{2}Y}|0\rangle = |\psi\rangle.$$

We can use the other Pauli matrices to add new potential eigenstates to $|\psi\rangle$ giving our circuit more expressivity.

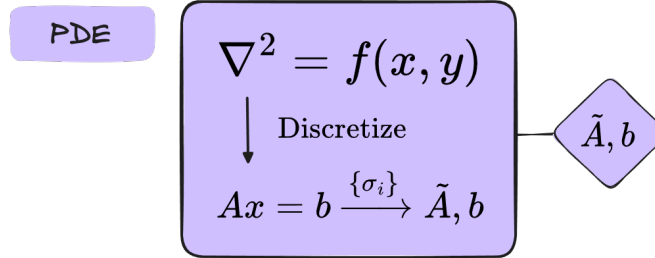
Chapter 3

Building a VQLS

Now that we have the motivation and the tool to solve the problem, we can begin building our VQLS. In the following sections I break down the four components of a VQLS. There are so many ways to build a VQLS. I have only detailed these sections for the application of VQLS for the Poisson problem. This is especially obvious for the first section about encoding the operators. There are plenty of books out there purposed for how to discretize PDEs so if you are interested in equations other than the Poisson then you will have to apply those texts here. In the rest of the sections they are generalizable to other problems you will just have to find the application yourself.

I have also built this clickable diagram on the cover of the book which outlines the VQLS algorithm in a visual manner. You can click each of the four main components of the diagram to bring you to the relevant chapter. For example, clicking the blue ansatz box will bring you to the ansatz section. Once there you can click the ansatz diagram to bring you back to the cover of the document. The diagram corresponds with the material in the following chapters and I hope after finishing this document you can use the diagram to see how each of the components fit together.

3.1 Constructing the operators



In my project `VQLSpsn` I am solving the 2D Poisson equation. So our goal for this section is to construct a Pauli string $\tilde{A}$ which represents the derivative information from our system of equations. I am going to first construct a 1D laplacian and decompose into Pauli matrices and then build the 2D laplacian with the 1D laplacian terms. This way the number of Pauli strings we end up evaluating is less than if we directly decomposed the 2D laplacian. You will see that this means we require less expectation values to complete a training loop. The 1D Poisson laplacian with zero dirichlet boundary conditions for N qubits is -2 along the diagonal and 1 along the off diagonal terms. For $N = 1$ per dimension we have two points along each axis so the grid spacing is given by $1/2$ and the 1D laplacian is given by

$$L_{1D} = \frac{1}{h^2} \cdot \begin{bmatrix} -2 & 1 \\ 1 & -2 \end{bmatrix} = \begin{bmatrix} -8 & 4 \\ 4 & -8 \end{bmatrix}.$$

We can decompose this matrix into a linear combination of Pauli matrices. The coefficients are computed using the equation $\frac{1}{2}Tr[L_{1D} * P]$ where P is a Pauli matrix. We end up getting

$$L_{1D} = -8I + 4X.$$

The 2D laplacian is given by the equation

$$A = (L_{1D} \otimes I) + (I \otimes L_{1D}).$$

Then if we multiply out the coefficients and write A as a linear combination of Pauli matrices we get

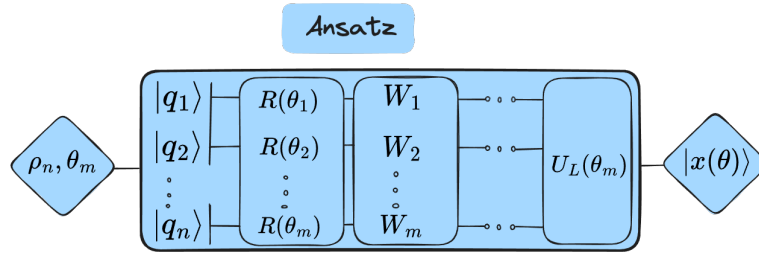
$$\tilde{A} = -16(II) + 4(IX) + 4(XI).$$

This is the final form of our A operator. So you see from the diagram that the final output from this step of the algorithm is the sum of Pauli operators and their coefficients in $\tilde{A}$ as well as the vector b . We can write this as a sum for each term k

$$\tilde{A} = \sum_k c_k P_k$$

where the weight of each term is given by c_k and the Pauli matrix by P_k .

3.2 Choosing an ansatz



If you remember from earlier, an ansatz is just a PQC that also has unparameterized gates. Ansatz prepares a quantum state which depends on parameters θ . When we first begin training our VQLS the final state produce results that don't align with our solution. However, after many iterations, a properly built VQLS will converge to a quantum state which represents an approximation to our solution. This process is guided by the cost function. The ansatz takes in initial qubits states ρ_n as well as the parameters from the rotation gates and outputs a final quantum state $|x(\theta)\rangle$.

$$|x(\theta)\rangle = U(\theta)|0\rangle.$$

Here $U(\theta)$ is the ansatz being applied to the zero state. The ansatz is defined by

$$U_l(\theta_l) = \prod_m e^{-i\theta_m H_m} W_m.$$

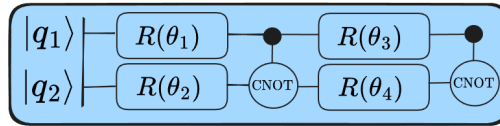
where W_m is the unparameterized unitary and H_m is the Hermitian operator. In qiskit the ansatz `real_amplitudes` has the form

$$U_{RA}(\theta) = \prod_m e^{-i\frac{\theta_m}{2} Y_m} \text{CNOT}_m.$$

where the Hermitian matrix is just the Pauli y matrix and the unparameterized unitary is the CNOT gate. If you are unfamiliar with the CNOT gate I would read through qiskit's documentation. When `real_amplitudes` is called for two qubits 0 and 1 for one layer

$$U_{RA}(\theta) = (\text{CNOT}_{0,1}) \cdot \left(e^{-i\frac{\theta_1}{2} Y_1} \right) \cdot \left(e^{-i\frac{\theta_2}{2} Y_2} \right).$$

Because the solution vector for our problem only has real values I have chosen this to be my ansatz. That way I know my circuit has enough expressivity but not too much that we are wasting computational resources. For $N_{\text{total}} = 2$ and $\text{depth} = 2$ if we use Penrose graphical notation to visualize `real_amplitudes` it would have four parameters total.



The total number of states in our system for $N_{\text{total}} = 2$ is $2^N = 4$ so we will have 4 coefficients in the final quantum state vector. The sum of squared amplitudes must equal one for quantum states. This removes one degree of freedom, and thus a 2 qubit system can be described with 3 independent parameters.

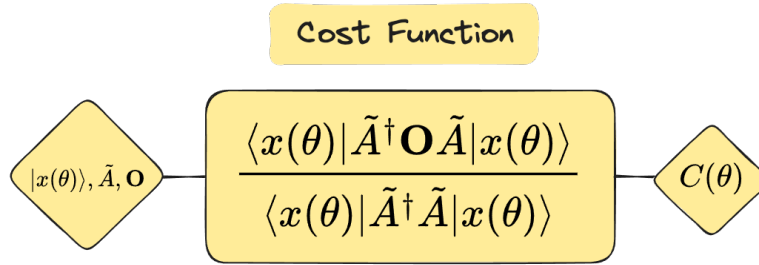
I want to add one final note to this section which may have not been obvious so far about VQLS. In the beginning of Ch.2 I defined the algorithm as preparing a quantum state $|x\rangle$ iteratively. Iteratively refers to the parameters inside of the ansatz being updated by the optimizer during each training loop. So the final solution is not just a quantum state of the actual solution x . The

final solution is a quantum state which depends on parameters θ which are optimized during the training loop. I will denote the trained parameters as θ_{opt} . The final solution is given by

$$U(\theta_{\text{opt}})|0\rangle = |x(\theta_{\text{opt}})\rangle.$$

This means after completing the training loop we obtain the optimal parameters θ_{opt} , which we plug into the ansatz, and apply again against a set of initialized qubits.

3.3 Building the cost function



Recall the variational principle. The expectation value of an eigenstate forms an upper bound for the ground state energy. Now instead of measuring the energy of the system we are measuring the overlap between our approximation and the original driving function. Really it can be said that we are measuring the overlap between our approximation and the solution vector. But this can be quite confusing if you then assume that we just have the solution vector from the start. We don't. In order to measure the overlap of the driving function, we make an approximation $|x(\theta)\rangle$, apply the laplacian to get a guess for the driving function $\tilde{A}|x(\theta)\rangle = |b_\theta\rangle$, and then measure the overlap between this guess and the original driving function b . The overlap is given by

$$|\langle b_\theta | b \rangle|^2 = \langle b_\theta | b \rangle \langle b | b_\theta \rangle.$$

This returns a real value between zero and one. When the vectors are overlapping the expression returns a value of one. When the vectors are orthogonal

to each other they return zero. It is standard to reverse this order and instead return zero when they are perfectly overlapping. We do this by subtracting the result from the identity matrix like so,

$$\langle b_\theta | (\mathbb{1} - |b\rangle\langle b|) | b_\theta \rangle.$$

We can expand the b_θ terms and group the inner product of b into an operator $\mathbf{O}$ and we get,

$$\langle x(\theta) | \tilde{A}^\dagger \mathbf{O} A | x(\theta) \rangle.$$

When $\tilde{A}$ is applied to $|x(\theta)\rangle$ the quantum hardware takes only the Pauli term P_k into account. Each qubit's measurement basis is changed to match the corresponding P_k term. The expectation value is taken with that Pauli term and then scaled by weight c_k which stretches the vector. Finally, all of these values are summed. This stretch can cause a couple of issues with the scaling of our final result. So we must normalize the the above equation in order to make a proper cost function. If we define the cost without normalizing the optimizer could do two things with the vector,

1. change the direction of $|b_\theta\rangle$ so it aligns with $|b\rangle$ (the correct way).
2. Shrink the length of $|b_\theta\rangle$ so the result also shrinks to zero (the wrong way).

So we must normalize the cost function by dividing by the square length of guess $|b_\theta\rangle$

$$C(\theta) = \frac{\langle b_\theta | (\mathbb{1} - |b\rangle\langle b|) | b_\theta \rangle}{\langle b_\theta | b_\theta \rangle}$$

Majority of the time we initialize the qubits of the ansatz circuit into the $|0\rangle$ state. We can expand the cost function again into

$$C(\theta) = \frac{\langle 0 | U^\dagger(\theta) \tilde{A}^\dagger (\mathbb{1} - |b\rangle\langle b|) \tilde{A} U(\theta) | 0 \rangle}{\langle 0 | U^\dagger(\theta) \tilde{A}^\dagger | \tilde{A} U(\theta) | 0 \rangle}.$$

Now the cost function is the normalized expectation value of a pre-defined operator with respect to $|b_\theta\rangle$

$$\mathbf{O}_G = \mathbb{1} - P_b$$

where P_b is called the "global" projection matrix. When we use $\mathbf{O}_G$ we are computing the global cost function. It works, but for systems with more qubits it often suffers from barren platues when the cost landscape is too flat for the optimizer to navigate. The local operator O_L was introduced to fix this issue. Instead of checking if the entire state $|b_\theta\rangle$ matches $|b\rangle$ we check one qubit at a time and average the score.

$$O_L = \mathbb{1} - \frac{1}{N} \sum_{q=1}^N U_b \left(|0\rangle \langle 0|_q \otimes \mathbb{1}_{\bar{q}} \right) U_b^\dagger$$

N is the total number of qubits q and U_b is a unitary Matrix which creates $|b\rangle$ out of the all-zero state

$$U_b |0\rangle^{\otimes n} = |b\rangle$$

So when $|b\rangle = |0\rangle$ we know $U_b = \mathbb{1}$. The term $\mathbb{1}_{\bar{q}}$ is the tensor product of 2×2 identity matrices applied to every qubit except the q^{th} qubit. For $q = 2$ and $U_b = \mathbb{1}$

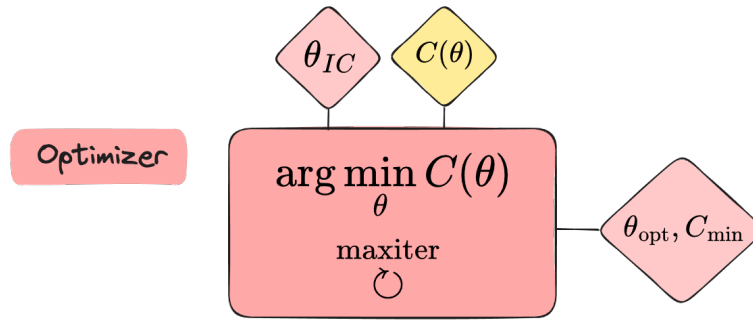
$$\begin{aligned} O_L &= \mathbb{1} - \frac{1}{2} \left(U_b \left(|0\rangle \langle 0|_1 \otimes \mathbb{1}_{\bar{1}} \right) U_b^\dagger + U_b \left(|0\rangle \langle 0|_2 \otimes \mathbb{1}_{\bar{2}} \right) U_b^\dagger \right) \\ O_L &= \mathbb{1} - \frac{1}{2} \left(U_b (|0\rangle \langle 0| \otimes \mathbb{1}) U_b^\dagger + U_b (\mathbb{1} \otimes |0\rangle \langle 0|) U_b^\dagger \right) \\ O_L &= \mathbb{1} - \frac{1}{2} \left(U_b \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} U_b^\dagger + U_b \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} U_b^\dagger \right) \\ O_L &= \mathbb{1} - \frac{1}{2} \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} = \mathbb{1} - \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{aligned}$$

for O_G with $q = 2$ and $U_b = \mathbb{1}$

$$\begin{aligned} O_G &= \mathbb{1} - P_G \\ &= \mathbb{1} - |b\rangle\langle b| \\ O_G &= \mathbb{1} - \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{aligned}$$

The math is very clear but still understanding how using the local operator makes a difference can be hard to see. The global operator asks: "Does the entire N state qubit perfectly match the state $|b\rangle$?" So if we have the state $|100001\rangle$ but need $|000000\rangle$, the global operator treats it as all or nothing and the cost landscape will remain unchanged. Unless you happen to randomly initialize your parameters θ incredibly close to the perfect answer, the cost function will return a value extremely close to one.

3.4 Choosing an optimizer



The optimizer controls how the parameters inside the ansatz in the cost function are updated during each training loop. It's goal is to minimize the value of the cost function over a pre-defined number of iterations.

$$\arg \min_{\theta} C(\theta).$$

There are many pre existing packages for optimizers in both qiskit and in scipy. Throughout my research I have mostly used scipy optimizers like

COBYLA which is a non-gradient descent technique. There are also quantum hardware specific optimizers in qiskit like **SPSA**. Most likely you will test a few optimizers from these packages for your project and compare the performance. To give intuition on how optimizers that use gradient descent make improvements take for example the global cost function which suffers from barren plateaus. The gradient is the derivative of the cost function with respect to your parameters. Because the variations in cost are small, the derivative evaluates to zero. A general rule for a gradient based optimizer is

$$\theta_{\text{new}} = \theta_{\text{old}} - (\text{learning rate} \cdot \text{gradient}).$$

So because the gradient is so close to zero the new parameters will only be slightly tweaked.

Chapter 4

The readout problem to be continued

The final output of the quantum circuit is a quantum state. The process of extracting the solution from the quantum state is referred to as the readout problem. If you want to solve a complex differential equation (like modeling airflow over a wing or fluid moving through a pipe), you usually have to calculate the answer over a massive "grid" of thousands or millions of tiny points to get an accurate result. Because of quantum superposition, a quantum computer can use just n qubits to represent 2^n data points simultaneously. For example, just 30 qubits can represent over a billion data points at the exact same time. On a regular computer, once you calculate that billion-point grid, you just open your hard drive, look at the memory, and read the answers.

A quantum computer does not let you do that. All those billion points are locked inside a fragile, unreadable quantum state. If you want to know the exact solution for the whole grid, you may use a process of running the exact same circuit millions of times and measuring it from every possible angle to slowly reconstruct the answer (a kind of tomography technique). Because the grid is so massive, this becomes impossibly complex and time-consuming. The time you save calculating the answer is completely wasted trying to read the answer.

ML methods were developed to take the quantum state solution and feed it

through a quantum neural network (QNN) to reduce the solution size before measurement. This way we can drastically simplify our result before measuring thereby reducing the cost of extraction. The methods are described in detail here: <https://arxiv.org/html/2411.14259v1> but I will give a short description.

Quantum Neural Network (QNN) is simply a series of new quantum gates appended directly to the end of your specific quantum algorithm circuit. The quantum state flows seamlessly from your solver directly into the QNN circuit $U(\boldsymbol{\theta})|\rho\rangle$ without ever being measured or turned into classical data.

The QNN systematically reduces the complexity of the state by concentrating the solution into a smaller subset of qubits. When the quantum state emerges from the end of the QNN, it is still a quantum state. However, its structure has been radically altered. Instead of the information being spread across an entangled 8-qubit or 12-qubit register, it is reduced to a 1 or 2-qubit register while the rest can be ignored. Now with this simple system we make a measurement.